# 현대오토에버 TCP·UDP 통신 포트폴리오

## 1. 프로젝트 개요

| 기간 / 인원 | 형식 / 플랫폼 | 기술 스택 | 담당 |
| --- | --- | --- | --- |
| 2025.10–2025.12 / 1인 | 네트워크 기술 데모 / PC (Windows) | Unity 6, C# | 네트워크 계층, 서버 시뮬레이션, 클라이언트 동기화 |

- 프로젝트 요약

  C#과 .NET 소켓을 활용한 TCP·UDP 기반 서버·클라이언트 통신 구현

  멀티플레이 게임을 실행 환경으로 활용해 데이터 송수신과 연결 관리 등 네트워크 개념을 학습

- 개발 배경

  서버 프로그래밍과 통신 처리의 동작 원리를 이해하기 위해 진행

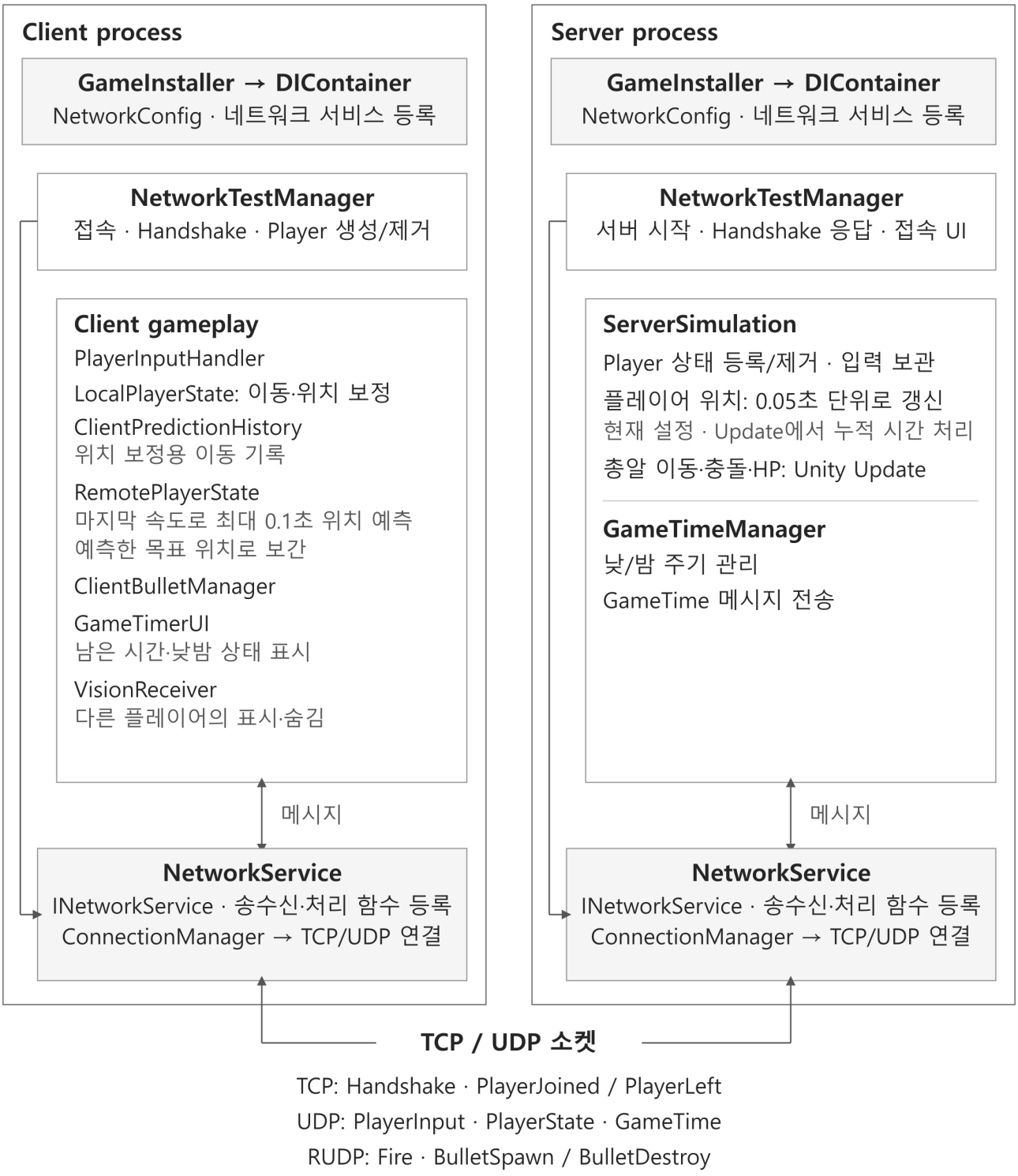
  처음에는 넷코드, 포톤과 같은 고수준의 네트워크 프레임워크를 사용하려했지만 확실한 개념 이해를 위해 .NET으로만 구현을 해보기로 함

## 2. 전체적인 아키텍처

- 동일한 Windows 빌드를 Server 또는 Client로 실행
- 게임 로직과 네트워크 계층을 분리해 게임 코드는 메시지 송수신만 요청하도록 구성

- 패킷 생성과 분할, 메시지 라우팅, TCP·UDP 전송은 네트워크 계층에서 처리

## Client / Server 실행 구조



## DIContainer의 역할

- 서비스 생성과 등록이 여러 script에 흩어지지 않도록 GameInstaller에서 필요한 서비스들을 한 번에 등록

```
Container.RegisterLazySingleton<IPacketProcessor>(c => new PacketProcessor(networkConfig, logger));
Container.RegisterLazySingleton<IConnectionManager>(c => new ConnectionManager(networkConfig, logger));
Container.RegisterLazySingleton<IMessageRouter>(c => new MessageRouter(logger));

var networkService = gameObject.AddComponent<NetworkService>();
networkService.Initialize(networkConfig, logger);
Container.RegisterSingleton<INetworkService, NetworkService>(networkService);
```

- 예를 들어, NetworkTestManager는 구체 클래스 대신 DIContainer에서 INetworkService와 같은 인터페이스를 조회해서 INetworkService의 기능을 호출함. 같은 인터페이스를 구현한 다른 서비스로 등록을 바꾸더라도, 사용하는 쪽의 호출 방식은 유지할 수 있도록 구성

## 네트워크 흐름

- NetworkTestManager는 서버 시작과 클라이언트 접속을 NetworkService에 요청하고, Handshake와 플레이어 입장, 퇴장 메시지를 처리
- NetworkTestManager와 인게임 코드는 INetworkService를 통해 서버 시작 및 접속과 메시지 송수신을 요청함. 서버와 클라이언트 모두 같은 NetworkService를 쓰지만 각 프로세스에서 별도의 객체로 실행
- NetworkService는 송신 메시지를 패킷으로 변환하고, ConnectionManager를 통해 전송, 수신 측에서는 복원된 메시지를 MessageRouter를 통해 해당 타입의 처리 함수로 전달

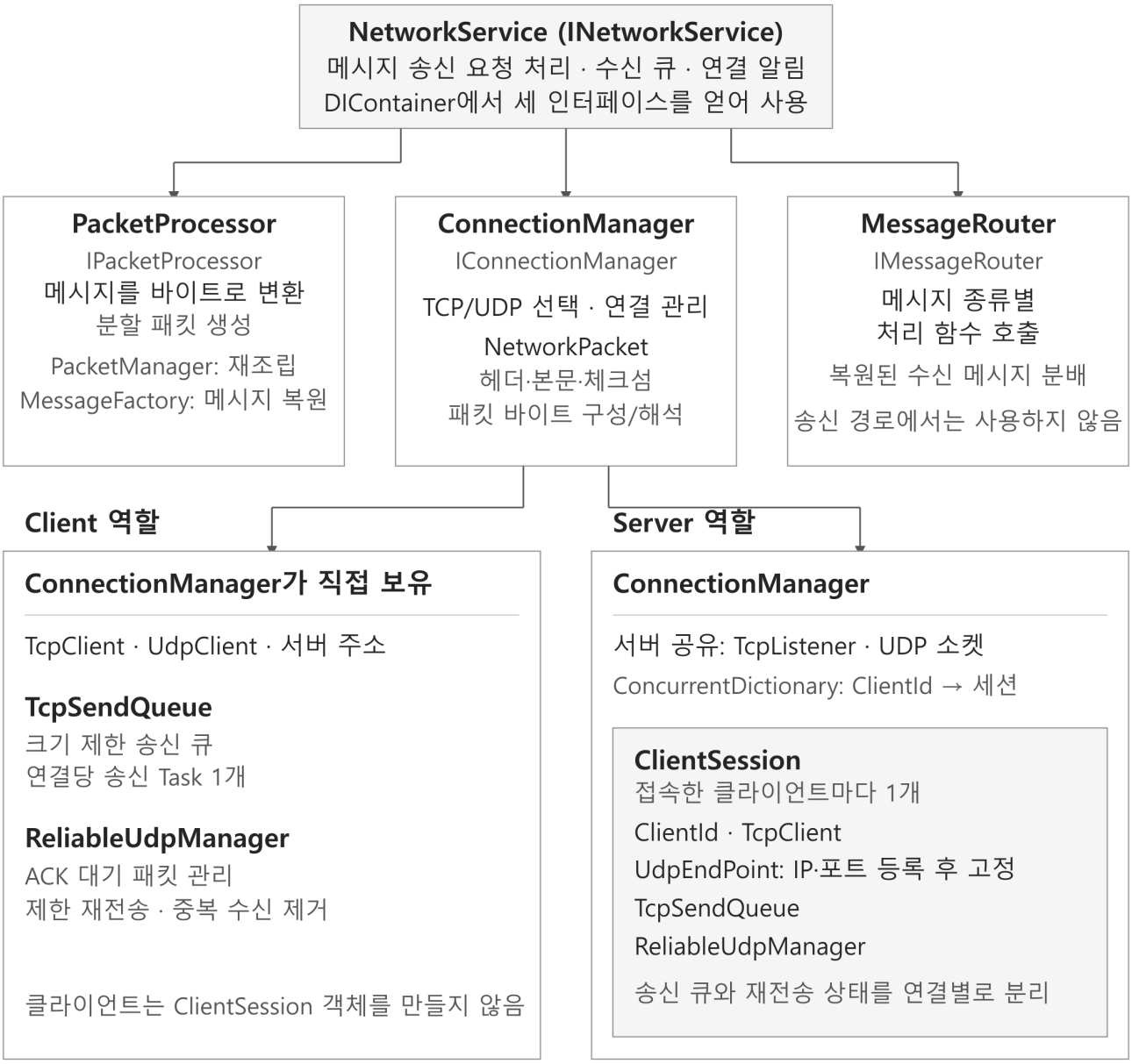
## TCP/UDP/RUDP 분리 기준

- Handshake와 플레이어 입장, 퇴장처럼 누락 없이 순서대로 처리해야 하는 메시지는 TCP로 처리
- 플레이어 입력과 상태, 인게임 시간처럼 계속 새 값이 전송되는 상태는 오래된 패킷을 다시 받는 것보다 최신 값이 중요한 만큼 UDP로 처리
- 총알 생성, 발사, 소멸처럼 한 번 누락되면 서버와 클라이언트의 상태 차이가 남는 사건은 UDP packet에 reliable 처리를 추가하여 RUDP로 처리

## 왜 RUDP를 사용하였나?

- 일반 UDP에는 누락된 패킷을 재전송하는 기능이 없어, 총알의 발사, 생성, 소멸 패킷에는 별도의 복구 처리가 필요
- 이것들을 TCP로 전달할 수도 있지만, 같은 연결에서 앞에 패킷이 누락되면 뒤에 도착한 것도 전달되기까지 기다림. 이를 피하기 위해 필요한 패킷에만 ACK와 재전송 로직을 추가하고, 다른 패킷은 앞선 패킷의 복구를 기다리지 않고 처리
- 대신 RUDP 자체에서는 전달 순서를 보장하지 않음. 총알 소멸 메시지가 생성보다 먼저 도착하는 경우에는 삭제 기록을 남겨 뒤늦은 생성을 막도록 처리

## 네트워크 구성과 연결별 데이터 관리




### NetworkService에서의 연결

- 패킷 변환, 소켓 연결, 메시지 전달을 한 클래스에서 모두 처리하지 않고 각각의 서비스로 분리함. NetworkService는 DIContainer에 등록된 세 서비스를 인터페이스로 가져와 연결함.

### 패킷 처리하기

- PacketProcessor는 메시지를 여러 패킷으로 나눠 전송하고, 수신 측은 원본 메시지를 복원함. 이때 패킷 Seq와 기준 Seq의 차이로 조각 순서를 계산
- 기존에는 번호 하나를 발급할 때마다 lock을 사용함. 번호 중복은 막았지만, 여러 메시지를 동시에 나누면 다른 메시지 번호가 중간에 끼어들 수 있었음. 번호가 연속되지 않아 수신할 때 패킷 조각의 위치를 잘못 계산해 복원에 실패하는 문제가 생김

- ReserveSeqNumbers()에서 메시지 하나에 필요한 번호들을 lock으로 할당하도록 수정함. 할당된 번호에 조각 순번을 더해 번호를 부여하고, 데이터 복사와 패킷 생성은 lock 밖에서 수행

```
private uint ReserveSeqNumbers(int count)
{
    lock (_seqLock)
    {
        uint first = unchecked(_seqNumber + 1);
        _seqNumber = unchecked(_seqNumber + (uint)count);
        return first;
    }
}
```

## 왜 ConcurrentQueue를 사용하는가

- 소켓에서 데이터를 받는 시점과 Unity가 게임 상태를 갱신하는 시점을 분리하기 위해 Queue를 사용함. 수신 즉시 플레이어 및 총알 처리 함수를 실행하지 않고, 받은 데이터를 보관했다가 NetworkService.Update()에서 처리
- 비동기 수신 작업의 추가와 메인 스레드의 꺼내기가 동시에 일어날 수 있어 일반 Queue 대신 ConcurrentQueue를 사용함. 접속, 종료 이벤트와 수신 패킷은 서로 다른 큐에 보관

```
private readonly ConcurrentQueue<ReceivedPacket> _incomingPackets = new();
private readonly ConcurrentQueue<QueuedConnectionEvent> _connectionEvents = new();
```

- 수신한 패킷에는 실제 송신자 ID와 현재 연결의 버전 번호를 함께 기록 ← 재접속 후 이전 연결에서 늦게 들어온 데이터를 구분하기 위한 정보

```
_incomingPackets.Enqueue(new ReceivedPacket
{
    Packet = packet,
    SenderId = actualSenderId,
    ConnectionVersion = connectionVersion
});
```

- Update()에서 접속, 종료 이벤트를 먼저 처리한 뒤 수신 큐에서 패킷을 꺼냄. 현재 연결의 버전과 다른 항목은 버리고, 유효한 패킷만 메시지로 복원해 MessageRouter로 전달
- ConcurrentQueue는 큐에 넣고 꺼내는 동작을 보호하며, 게임 로직의 실행 위치는 Update()가 메인 스레드로 제한함. 처리 개수 제한은 한 번에 실행하는 양을 줄이지만, 수신량이 계속 처리량보다 많으면 대기가 쌓일 수 있어 따로 경고 처리

## ConnectionManager에서의 전송

- 게임 코드가 메시지마다 소켓을 선택하지 않도록 전송 분기를 ConnectionManager로 통일 SendPackets()는 패킷을 byte 배열로 직렬화하고, 패킷 타입에 따라 TCP 또는 UDP 송신 함수에 전달함.

```
public void SendPackets(NetworkPacket[] packets)
{
    if (packets == null || packets.Length == 0) return;

    foreach (var packet in packets)
    {
        try
        {
            var data = packet.Serialize();

            if (RequiresTcp(packet.Header.Type))
            {
                SendViaTcp(data, packet.TargetId);
                _stats.TcpPacketsSent++;
            }
            else
            {
                bool reliable = packet.Header.Flags.HasFlag(PacketFlags.Reliable);
                SendViaUdp(data, packet.TargetId, reliable, packet.Header.SeqNumber);
                _stats.UdpPacketsSent++;
            }
        }
        catch (Exception ex)
        {
            _logger.LogException(ex, "SendPacket");
        }
    }
}
```

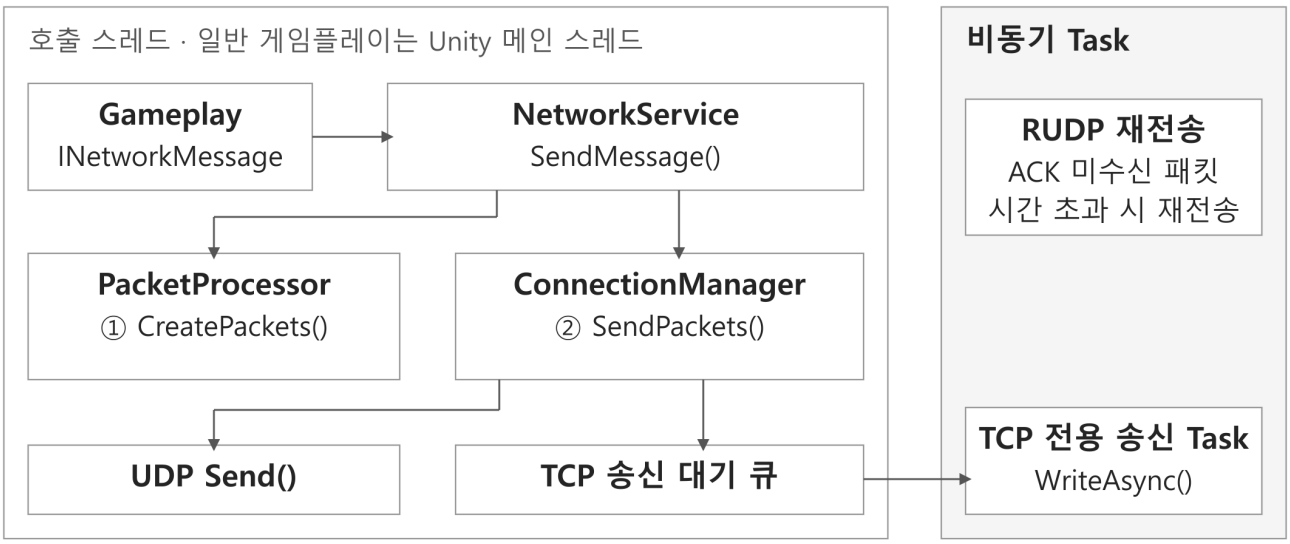
- UDP 경로에서는 Reliable 플래그와 Seq를 함께 넘겨 재전송이 필요한 패킷을 구분함.
- 각 클라이언트는 서버 하나와 통신하므로, ConnectionManager가 서버로 보낼 TCP 데이터와 RUDP 재전송 기록을 관리함.

## ClientSession의 역할

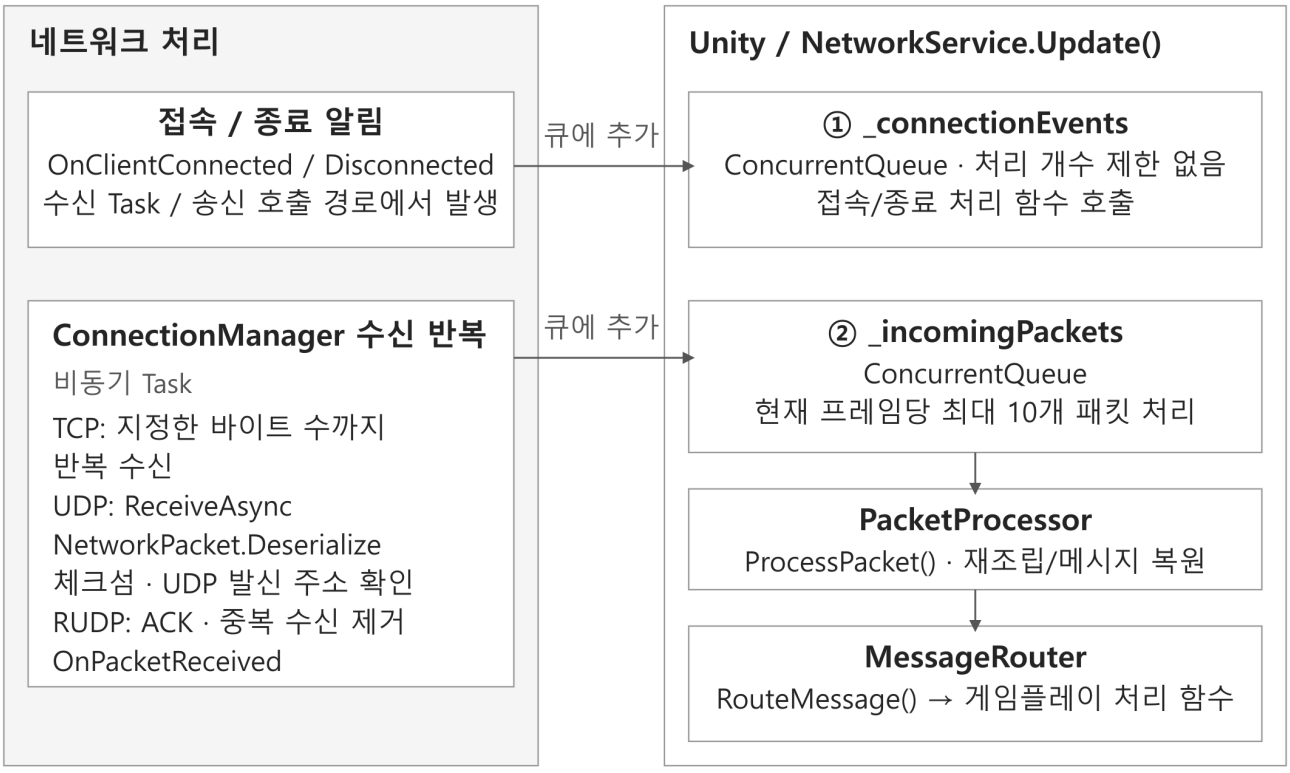
- 하나의 서버는 여러 클라이언트와 통신하므로, 접속자마다 ClientSession을 만들어 각 클라이언트와의 연결과 전송 기록을 따로 관리
- 각 세션은 해당 클라이언트로 보낼 TCP 데이터를 송신 큐에 보관하고, 전송 작업이 순서대로 진행되도록 함. ACK를 기다리는 RUDP 패킷과 재전송 기록도 세션별로 관리하며, 서버의 UDP 소켓은 큐를 거치지 않고 공통으로 사용
- TCP에서는 전송 작업을 클라이언트별로 나눠, 한 클라이언트로의 전송이 늦어져도 다른 클라이언트의 전송이 그 작업의 완료를 기다리지 않도록 함. ACK를 받으면 해당 세션에서만 확인된 패킷을 제거

# 3. TCP 패킷 구분과 동시 송신 처리

### 송신




### 수신




UDP 등록과 ACK처럼 연결·재전송 관리에 쓰이는 패킷은 전송 계층에서 처리하며, 게임 로직으로 전달하는 수신 큐에는 넣지 않음.

- TCP는 바이트가 연속해서 전달되는 방식이므로 ReadAsync() 한 번으로 패킷 하나를 전부 못받는 경우도 생김. 한 패킷을 여러 번에 나눠 받거나, 여러 패킷의 데이터를 한꺼번에 받는 경우도 존재

- 패킷 앞에 데이터 길이를 나타내는 4바이트를 붙여 전송함. 이 길이 정보와 실제 데이터를 합친 전송 단위를 프레임으로 사용함. 수신 측은 길이 정보를 먼저 읽고, 그 크기만큼 데이터를 모아 패킷 하나를 완성
- 패킷의 길이 정보와 실제 데이터는 각각 ReadExactAsync()로 필요한 크기만큼 읽음. 일부만 받았다면 이미 읽은 위치 뒤부터 남은 부분을 이어서 읽고, 전부 받기 전에 연결이 끝나면 남은 패킷은 넘기지 않음.

```
private async Task<bool> ReadExactAsync(NetworkStream stream, byte[] buffer, int offset, int count, CancellationToken ct)
{
    int totalRead = 0;
    while (totalRead < count)
    {
        int bytesRead = await stream.ReadAsync(buffer, offset + totalRead, count - totalRead, ct);
        if (bytesRead == 0)
            return false;

        totalRead += bytesRead;
    }
    return true;
}
```

## 서버-클라이언트 연결마다 하나의 비동기 태스크

같은 TCP 연결에 여러 작업이 동시에 송신을 요청할 수 있음. 각 작업이 소켓에 직접 쓰는 대신, 완성된 프레임을 큐에 넣고 연결마다 하나의 송신 작업이 차례로 전송하도록 구성

- 전송 순서는 함수 호출을 시작한 시각이 아니라 큐에 들어간 순서로 정함

```
while (true)
{
    await _frameAvailable.WaitAsync(cancellationToken).ConfigureAwait(false);

    byte[] frameData;
    lock (_queueLock)
    {
        if (_frames.Count == 0)
        {
            if (!_acceptingFrames)
                break;

            continue;
        }

        frameData = _frames.Dequeue();
        _queuedBytes -= frameData.Length;

        if (_frames.Count <= _maxQueuedFrames / 2 &&
            _queuedBytes <= _maxQueuedBytes / 2)
        {
            Volatile.Write(ref _queueFullLogged, 0);
        }
    }

    await _stream.WriteAsync(
            frameData,
            0,
            frameData.Length,
            cancellationToken)
        .ConfigureAwait(false);
}
```

- 큐에 데이터를 넣고 꺼내는 작업과 대기 데이터의 바이트 합계 변경만 같은 lock으로 보호함. 소켓에 넣는 작업은 lock 밖에서 수행해, 전송이 느려져도 다른 송신 요청이 큐에 들어가는 것까지 막지 않도록 함.

- 큐가 비어 있으면 반복해서 확인하지 않고 대기, 전송할 데이터가 들어오면 이어서 처리

### 송신 작업에서 제한이 필요한 이유

송신 작업을 하나로 제한해도 전송 속도보다 요청이 빠르면 큐에 데이터가 계속 쌓임. 대기 개수만 제한하면 큰 데이터의 메모리 사용량을 제어하기 어렵고, 용량만 제한하면 데이터의 양이 많아질 수 있어 둘 다 적용

```
lock (_queueLock)
{
    if (!_acceptingFrames || Volatile.Read(ref _stopRequested) != 0)
    {
        result = TcpFrameQueueResult.Disposed;
    }
    else if (_frames.Count >= _maxQueuedFrames ||
             frameData.Length > _maxQueuedBytes - _queuedBytes)
    {
        result = TcpFrameQueueResult.QueueFull;
    }
    else
    {
        _frames.Enqueue(frameData);
        _queuedBytes += frameData.Length;
        _frameAvailable.Release();
        return TcpFrameQueueResult.Queued;
    }

    queuedFrames = _frames.Count;
    queuedBytes = _queuedBytes;
}
```

- 전송이 밀릴 때 데이터가 계속 쌓이지 않도록, 송신 큐에서 대기하는 데이터의 개수와 용량을 제한
- 여러 요청이 동시에 들어와도 제한을 넘지 않도록 확인하고 큐 삽입과 용량 갱신을 같은 lock 안에서 처리함.
- 제한을 넘으면 새 요청을 거부하고 해당 연결을 종료. 이때 아직 보내지 못한 데이터는 전달되지 않을 수 있음.

# 4. RUDP 재전송과 중복 처리

## ACK가 없는 패킷만 재전송

- Reliable 패킷을 보낼 때는 패킷 번호, 데이터, 전송 시각을 함께 보관함. 해당 번호의 ACK를 받으면 기록을 지우고, 일정 시간 동안 ACK가 없는 패킷만 다시 전송함. 재전송 횟수에도 제한을 두었으므로, 끝내 ACK를 받지 못한 패킷의 전달까지 보장하지는 않음.

```
public List<byte[]> GetPacketsToRetry()
{
    lock (_lock)
    {
        var packetsToRetry = new List<byte[]>();
        double currentTime = _stopwatch.Elapsed.TotalSeconds;

        foreach (var kvp in _pendingPackets)
        {
            var packet = kvp.Value;

            if (currentTime - packet.LastSendTime > _retryTimeout && packet.RetryCount < _maxRetries)
            {
                packet.LastSendTime = currentTime;
                packet.RetryCount++;
                packetsToRetry.Add(packet.Data);
            }
        }

        return packetsToRetry;
    }
}
```

## 같은 패킷이 여러 번 도착하는 경우

- RUDP에서 패킷은 정상적으로 도착했어도 ACK가 누락되면 송신 측에서 같은 패킷을 다시 보냄. 이를 그대로 처리하면 발사 같은 동작이 중복 실행될 수 있어, 수신한 Seq를 기록해 이미 처리한 패킷을 구분
- 중복 패킷은 게임 로직에 안넘기지만 ACK는 다시 보냄. ACK까지 생략하면 송신 측이 계속 응답을 기다리며 재전송하기 때문
- 중복 확인에는 HashSet을 사용함. 평균 O(1)에 중복 여부 조회 가능
- HashSet만으로는 어떤 기록을 먼저 지워야 하는지 알 수 없어, 저장 순서를 큐에 함께 남김. 서로 다른 Seq 2,048개를 넘으면 가장 먼저 저장한 기록을 큐와 HashSet에서 제거

```
private readonly HashSet<uint> _receivedSeqs = new();
private readonly Queue<uint> _receivedSeqOrder = new();
```

- 중복 확인, 히스토리 추가, 오래된 기록 제거는 하나의 lock 안에서 처리함. 같은 Seq가 동시에 들어와도 두 작업이 모두 처음 받은 패킷으로 판단하지 않도록 함

```
public bool TryAcceptIncoming(uint seqNumber)
{
    lock (_lock)
    {
        if (_receivedSeqs.Contains(seqNumber))
            return false;

        _receivedSeqs.Add(seqNumber);
        _receivedSeqOrder.Enqueue(seqNumber);

        while (_receivedSeqOrder.Count > MaxTrackedReceivedSeqs)
        {
            uint oldestSeq = _receivedSeqOrder.Dequeue();
            _receivedSeqs.Remove(oldestSeq);
        }

        return true;
    }
}
```

### 총알 소멸 뒤에 생성 메시지가 도착하는 경우

총알 생성 메시지를 먼저 보냈더라도, 수신 측에는 소멸 메시지가 먼저 도착할 수 있음. 생성과 소멸은 서로 다른 메시지라 중복 패킷 검사로는 걸러지지 않음. 따라서 이미 소멸한 총알의 ID를 저장해 두고, 해당 총알의 생성 메시지가 뒤늦게 도착하면 무시하도록 함

- ClientBulletManager에서 총알 소멸 메시지를 받으면 해당 총알의 BulletId를 HashSet에 저장. 생성 메시지가 아직 도착하지 않았더라도, 나중에 해당 총알이 생성되지 않도록 ID를 저장함.
- 총알 생성 메시지를 받으면 해당 BulletId가 HashSet에 있는지 확인함. 있으면 이미 소멸한 총알이므로 생성 요청을 무시함.
- 소멸한 총알의 ID는 최대 2,048개만 보관하고, 초과하면 먼저 저장한 ID부터 제거함. 제거된 ID의 생성 메시지가 뒤늦게 도착하면 이미 소멸한 총알이 다시 생성될 수 있음.
- 이처럼 다시 생성되거나 소멸 메시지를 받지 못한 총알이 화면에 계속 남지 않도록 클라이언트에서도 생성 후 설정된 수명이 지나면 제거함. 다만 잘못된 재생성을 막거나 서버의 소멸 시각과 정확히 맞추는 처리는 아니라 보완이 필요

## 5. TCP 접속자와 UDP 송신 주소 연결

TCP로 접속한 클라이언트가 UDP도 함께 사용하므로, 서버는 UDP 패킷을 어느 접속자가 보냈는지 구분해야 함. 하지만 패킷 안의 ClientId는 보내는 쪽에서 바꿀 수 있어, 이 값만으로 접속자를 판단할 수 없음.

이를 보완하기 위해 TCP 연결로 전달한 토큰을 UDP로 돌려받아, 해당 접속자가 사용하는 UDP 주소(패킷을 보낸 IP 주소와 포트)를 등록하도록 변경

### TCP로 받은 토큰을 UDP 주소 등록에 사용

- 서버는 세션마다 CSPRNG로 생성한 예측하기 어려운 64비트 토큰을 TCP로 전달함. 클라이언트가 UDP로 돌려보낸 토큰이 해당 세션에 발급한 값과 일치할 때만 송신 주소를 등록하도록 함.
- 주소 등록이 끝나기 전에 재접속하거나 서버를 다시 시작하면, 이전 연결에서 진행하던 작업이 뒤늦게 처리될 수 있음. 토큰 확인말고도 서버 실행을 구분하는 번호와 세션 객체도 비교해 현재 연결의 등록 작업일 때만 주소를 반영
- 다른 세션이 이미 사용하는 주소는 등록하지 않도록 해서 하나의 UDP 주소가 여러 세션에 등록되는 경우를 걸러냄. 같은 세션이 등록된 주소로 다시 요청하는 것은 허용하되, 접속 중 주소를 바꾸는 요청은 거부
- 검사할 때 유효했던 세션이 실제 주소를 저장하기 전에 종료될 수도 있음. 확인과 저장을 따로 처리하면 이런 변경을 놓칠 수 있어, 세션 확인·주소 중복 검사·주소 등록을 같은 lock 안에서 수행함.

```
lock (_transportLifecycleLock)
{
    if (IsCurrentTransportLocked(transportGeneration, TransportMode.Server) &&
        _udpRegistrations.TryGetValue(clientId, out var registration) &&
        registration.Token == token &&
        _sessions.TryGetValue(clientId, out var currentSession) &&
        ReferenceEquals(currentSession, registration.Session))
    {
        bool endpointAlreadyOwned = false;
        foreach (var existingSession in _sessions.Values)
        {
            if (!ReferenceEquals(existingSession, currentSession) &&
                existingSession.UdpEndPoint?.Equals(remoteEndPoint) == true)
            {
                endpointAlreadyOwned = true;
                break;
            }
        }

        if (!endpointAlreadyOwned &&
            (currentSession.UdpEndPoint == null ||
             currentSession.UdpEndPoint.Equals(remoteEndPoint)))
        {
            currentSession.UdpEndPoint = remoteEndPoint;
            currentSession.UpdateActivity();
            registeredSession = currentSession;
            pendingReliablePackets = registration.PendingReliableUdp.Drain();
            registered = true;
        }
    }
}
```

- 실제 UDP 패킷을 받을 때는 등록된 송신 주소로 세션을 찾은 뒤, 헤더의 ClientId가 그 세션과 일치하는지도 확인함. 주소와 ClientId가 서로 다른 접속자를 가리키는 패킷은 무시함

### 주소 등록 전에는 필요한 패킷만 보관

- TCP 접속 직후에는 UDP 주소 등록이 아직 끝나지 않았을 수 있음. 이때 Reliable 패킷은 임시 보관하고, 등록이 끝나 실제 송신을 시도할 때부터 ACK 대기를 시작함. 보내기도 전에 재전송 시간이 흐르는 것을 피하기 위한 처리
- 이동 상태처럼 재전송을 안하는 UDP 패킷은 보관하지 않음. 등록 이후 최신 상태를 보내면 되므로, 이전 상태를 뒤늦게 전송하지 않도록 함.

## 6. 소스 코드

소스 코드는 같은 저장소의 Scripts 폴더에 C# 파일(.cs)만 모아 두었음.
각 스크립트는 문서에서 설명한 클래스 이름으로 찾아볼 수 있음.
실행용 Unity 프로젝트는 포함하지 않은 코드 열람용 자료임.